# Sorting - 2

**Agenda:**

1. Smallest Number
2. Count Sort
3. Merge 2 sorted arr.
4. Merge Sort algorithm

## ① Finding Smallest no.

~~(Count Sort)~~
(Count Sort)

(if digits are basic 0-9)

or range of digits is limited and less $10^7$.

```
Public class SmallestNumber {
    Public static int[] findSmallest(int[] A) {
        int freq[] = new int [10]; //to store freq. of digits 0-9
        for (int x : A) {
            freq[x]++ ;
        }
        int [] result = new int [A.length];
        int idx = 0;
        for (int d=0 ; d <=9 ; d++) {    //Iterate through digits 0-9
            for (int i =0 ; i < freq[d] ; i++) {
//Fill digits based on freq. →    result [idx] = d;
                idx ++ ;
            }
        }
        return result;
    }
}
```

## ② Handling Negative Numbers

```
Publ... — —
P ... — — —
int min = A[0] , max = A[0]
for (int x : A) {
        if (x < min) min = x ;
        if (x > max) max = x ;
int range = max - min + 1
    int [] freq = new int [range] ;
for (int x : A) {
    freq [x - min] ++ ;
}
```

☆ rest like normal count sort → digit_range = range

## ③ Merging 2 sorted arrays

~~Input:~~

Given 2 sorted arrays A, B / merge B and A as one sorted array and return it.

(linear TC is expected)

T.C = O(N+M)

Constraints:

$-2\times10^9 <= A[i] <= 2\times10^9$

$B[i]$

$1 <= |A|, |B| <= 5\times10^4$

ex: A = [4, 7, 9]
B = [2, 11, 19]
out = [2, 4, 7, 9, 11, 19]

i → A
j → B
k → ans

Sol.

```
public class sol {
    public int[] Soln( int[]A, int[]B)
        int M = A.length;
        int N = B.length;
        int[] ans = new int [M+N]
          int i=0 ;  int j= 0;  int k =0;

    while(i<M  &&  j<N) {
            if ( A[i] < B[j]) {
                    ans[k] = A[i]
                     k++ ;    i++;
            else {
                    ans[k] = B[j]
                    k++ ;    j++;
            }
        }
        while (i< M ){
                ans [k]  =A[i]
                    k++;   i++;
            }
        while (j< N){
            ans [k]= B[j]
                k++;    j++ ;
            }
        return ans;
    }
}
```

# Merge Sort Algorithm (Divide and Conquer)

1. Divide : split the array into 2 halves at the mid point.
2. Conquer: Recursively sort both halves.
3. Combine: Merge the 2 sorted halves back together.

```
Public class MSA {
    Public void sort (int[] A, int l, int r) {
        if (l >= r) {
            return;
        }
        int mid = (l+r)/2;
        sort (A, l, mid);
        sort (A, mid+1, r);

        merge (A, l, mid, r);
    }

    private void merge (int[] A, int l, int mid, int r) {
        int[] ans = new int [r-l+1];
        int i = l;
        int j = mid+1
        int k = 0;

        while (i <= mid && j <= r) {
            if (A[i] <= A[j]) {
                ans [k++] = A[i++];
            }
            else { ans[k++] = A [j++]; }
        }

        while (i <= mid) { ans[k++] = A[i++]; }

        while (j <= r) { ans[k++] = A [j++]; }

        for (int p = 0; p < ans.length; p++) {
            A [l+p] = ans [p];
        }
    }
}
```

// Base case, if there is 1 element or invalid range

// Recursively sort first & second have

// merge sorted halves

// Temp. arr. to store merged result

// Compare and Pick the smaller one

// copy remaining element of left half, if any

// same ---

// copy merged parts back into original arr.

same as merging 2 sorted arr.